# Step 2

**200 OK**

The logic is the server sends 200 OK when the requested file is found and is accessible.

The HTTP request message I will use to test it is curl -i http://localhost:9090/test.html

**304 Not Modified**

The logic is the server sends 304 Not Modified when the file has not changed since the date in the If-Modified-Since header.

The HTTP request message I will use to test it is curl -i -H "If-Modified-Since: Sun, 30 Aug 2026 12:00:00 GMT" http://localhost:9090/test.html

**403 Forbidden**

The logic is the server sends 403 Forbidden when the client tries to access a file that is not allowed.

The HTTP request message I will use to test it is curl -i http://localhost:9090/forbidden.html

**404 Not Found**

The logic is the server returns 404 Not Found when the file asked for does not exist.

The HTTP request message I will use to test it is curl -i http://localhost:9090/missing.html

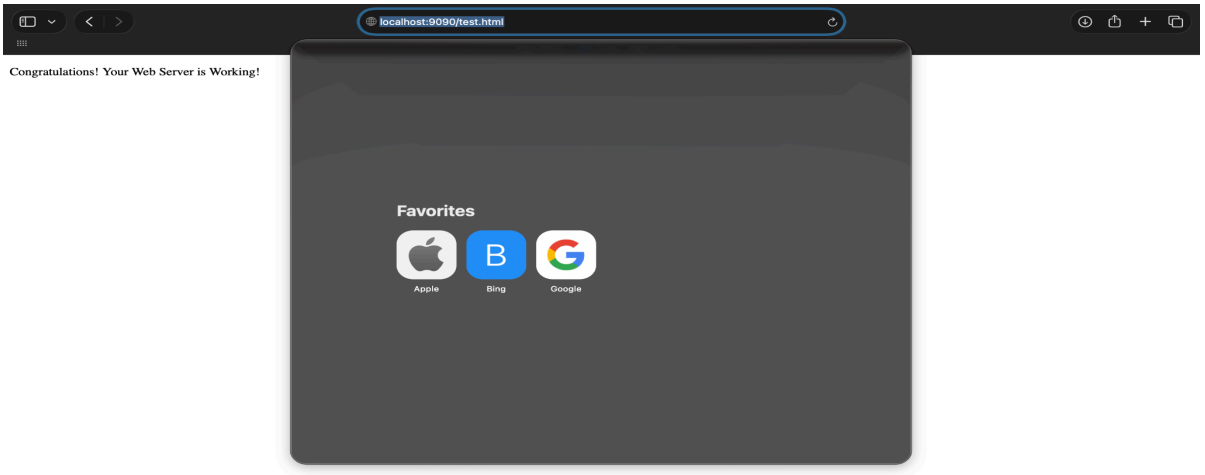
**505 HTTP Version Not Supported**

The logic is the server returns 505 HTTP Version Not Supported if the client sends a request using an HTTP version the server cannot handle.

The HTTP request message I will use to test it is curl -i --http1.0 http://localhost:9090/test.html

# Step 3

b) http://localhost:9090/test.html




c)

**200 OK**

The test procedure I used was curl -i http://localhost:9090/test.html

```
(base) arbindkhaira@Arbinds-MacBook-Pro-6 miniproject % curl -i http://localhost:9090/test.html
HTTP/1.1 200 OK
Content-Length: 308
Content-Type: text/html; charset=utf-8
Connection: close

<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <title></title>
  <meta name="author" content="">
  <meta name="description" content="">
  <meta name="viewport" content="width=device-width, initial-scale=1">

</head>

<body>

  <p>Congratulations! Your Web Server is Working!</p>

</body>

</html>
```

### 304 Not Modified

The test procedure I used was curl -i -H "If-Modified-Since: Sun, 30 Aug 2026 12:00:00 GMT" http://localhost:9090/test.html

```
(base) arbindkhaira@Arbinds-MacBook-Pro-6 miniproject % curl -i -H "If-Modified-Since: Wed, 30 Aug 2026 12:00:00
GMT" http://localhost:9090/test.html
HTTP/1.1 304 Not Modified
Content-Length: 0
Content-Type: text/plain; charset=utf-8
Connection: close
```

### 403 Forbidden

The test procedure I used was curl -i http://localhost:9090/forbidden.html

```
(base) arbindkhaira@Arbinds-MacBook-Pro-6 miniproject % curl -i http://localhost:9090/forbidden.html
HTTP/1.1 403 Forbidden
Content-Length: 14
Content-Type: text/plain; charset=utf-8
Connection: close

403 Forbidden
```

### 404 Not Found

The test procedure I used was curl -i http://localhost:9090/missing.html

```
(base) arbindkhaira@Arbinds-MacBook-Pro-6 miniproject % curl -i http://localhost:9090/missing.html
HTTP/1.1 404 Not Found
Content-Length: 14
Content-Type: text/plain; charset=utf-8
Connection: close

404 Not Found
```

### 505 HTTP Version Not Supported

The test procedure I used was curl -i --http1.0 http://localhost:9090/test.html

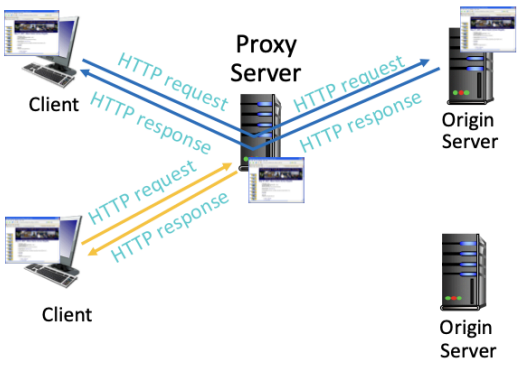
```
(base) arbindkhaira@Arbinds-MacBook-Pro-6 miniproject %  curl -i --http1.0 http://localhost:9090/test.html
HTTP/1.1 505 HTTP Version Not Supported
Content-Length: 31
Content-Type: text/plain; charset=utf-8
Connection: close

505 HTTP Version Not Supported
```

# Step 4: Proxy server Implementation

## Proxy Server

What is the difference between request handling between a web server and a proxy server?

The main difference is that the proxy server acts as both a client and server. It accepts client requests as a server and then makes requests towards the origin server to obtain the file (hence acting as the client in the second case.). We used the model below to implement our proxy server.




**The proxy server acts as an intermediary between the client and the origin web server. The client does not know the address or port of the origin server and sends all HTTP requests directly to the proxy server. The origin server's address is predefined in the proxy program as localhost on port 9090.**

For example, the client requests test.html using:

```
curl -i http://localhost:8888/test.html
```

The client only specifies the proxy server's port, 8888. The proxy is responsible for communicating with the origin server on port 9090.

### Step-by-Step Operation

**Receive the client request**
The proxy server listens for client connections on port 8888. When the client requests a file, the proxy accepts the connection and reads the HTTP request.
Example request:

```
GET /test.html HTTP/1.1
Host: localhost:8888
```

1. **Extract the requested filename**
   The proxy examines the request line and extracts the requested path. For the previous request, the extracted path is:
   /test.html

2. **Determine the cache location**
   The proxy maps the requested path to a file inside its local cache directory. For example:
   /test.html → cache/test.html
3. The proxy creates the cache directory automatically if it does not already exist.
4. **Check whether a cached copy exists**
   The proxy checks whether cache/test.html exists.
   Two possible situations can occur:
   - The requested file is not in the cache.
   - A cached copy of the requested file already exists.

**Handle a cache miss**
If cache/test.html does not exist, the proxy sends a normal HTTP GET request to the predefined origin server:

```
GET /test.html HTTP/1.1
Host: 127.0.0.1:9090
Connection: close
```

5. If the origin server returns 200 OK, the proxy:
   - Extracts the response body.
   - Saves the file as cache/test.html.
   - Sends the downloaded file to the client with a 200 OK response.
6. Responses such as 403 Forbidden or 404 Not Found are forwarded to the client but are not stored in the cache.

**Handle an existing cached file**
If cache/test.html already exists, the proxy obtains its last-modified time from the file system. It converts this time into the HTTP date format, such as:
Mon, 03 Aug 2026 21:45:53 GMT

7. **Send a conditional request to the origin server**
   The proxy sends a new request to the origin server containing the cached file's modification date in the If-Modified-Since header:

```
GET /test.html HTTP/1.1 Host: 127.0.0.1:9090

If-Modified-Since: Mon, 03 Aug 2026 21:45:53 GMT
Connection: close
```

8. This asks the origin server whether test.html has changed since the cached copy was saved.

**Process a 304 Not Modified response**
If the origin file has not changed, the origin server responds with:

```
HTTP/1.1 304 Not Modified
```

9. A 304 response does not contain the file body. Therefore, the proxy reads cache/test.html and sends the cached content to the client as a complete 200 OK

response.
This avoids downloading the same unchanged file again from the origin server.

10. **Process a 200 OK response**
    If the origin file has changed, the origin server responds with 200 OK and includes the updated file.
    The proxy then:
    - Downloads the updated response body.
    - Replaces the old cache/test.html file with the new version.
    - Sends the updated file to the client as a 200 OK response.
11. **Close the connections**
    After sending the response, the proxy closes the client connection. The origin connection is also closed after its complete response has been received.

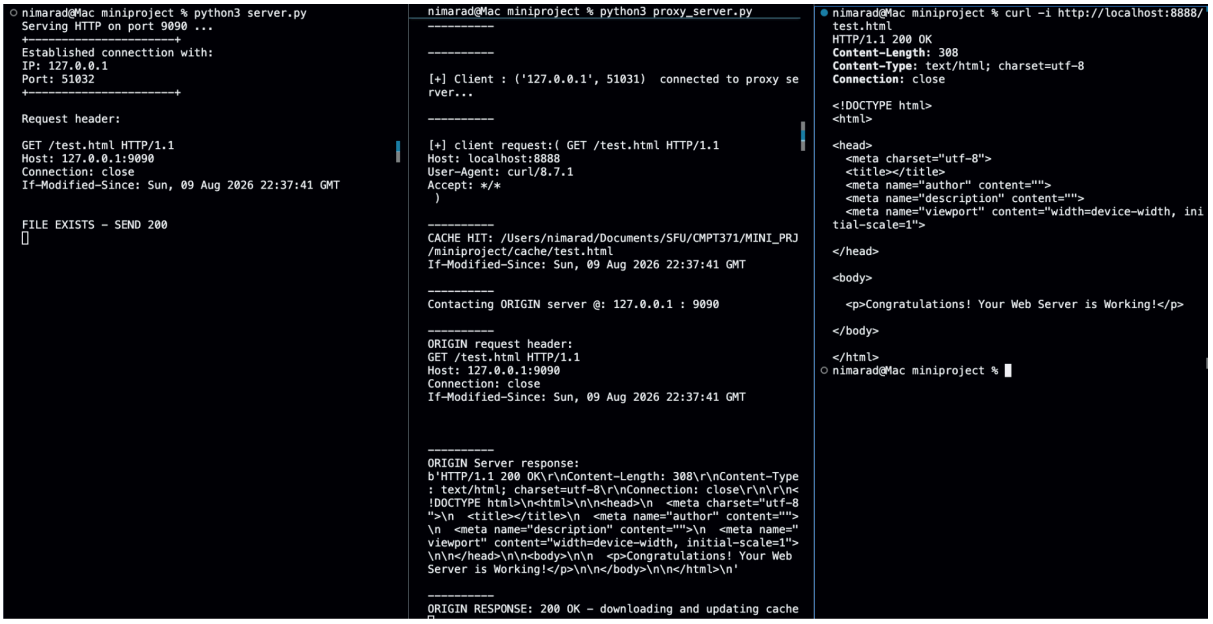
This caching process reduces unnecessary file transfers while ensuring that the client receives the most recent available version of the requested file.

# Testing the Proxy Server Implementation

To verify the proxy functionality, initialize three separate terminal instances:
**Terminal 1 (Origin Server on port 9090):**
`python3 server.py`
**Terminal 2 (Proxy Server on port 8888):**
`python3 proxyserver.py`

**Terminal 3 (Client Request):**
`curl -i http://localhost:8888/test.html`
**Expected System Behavior:**

12. **Initial Request Processing**
    The origin server provides a 200 OK response, and the proxy subsequently generates the local cache/test.html file.
13. **Subsequent Request Handling**
    The proxy utilizes the If-Modified-Since header to transmit the modification timestamp of the cached version.
14. **Unchanged Content Verification**
    When the origin returns 304 Not Modified, the proxy serves the existing cached copy to the client as a 200 OK response.
15. **Updated Content Synchronization**
    If the file has changed, the origin sends a 200 OK response, prompting the proxy to refresh the local cached file. 5. Any 403 Forbidden or 404 Not Found responses are not preserved in the cache.




This caching process reduces unnecessary file transfers while ensuring that the client receives
the most recent available version of the requested file.

# Step 4: Multi-threaded Server Implementation

**Why thread-server.py is multi-thread server?**

The initial `server.py` implementation was strictly single-threaded, meaning it processed a client's request entirely before it could accept a subsequent connection.
To enable concurrent processing, we utilized the `threading` module to relay each new client connection to a dedicated thread, allowing multiple requests to be handled simultaneously.

**2. Encapsulate request processing logic**

```
def handle_client(client_connection, client_address):
```

This function encapsulates all the necessary request-handling operations:

- 
- Parses the incoming HTTP request data.
- Validates the specified HTTP version.
- Manages response codes including 200, 304, 403, 404, and 505.
- Retrieves the file and transmits it to the client.
- Terminates the client socket connection.

A `finally` block is used to guarantee the connection is terminated regardless of errors:

```
finally:
    client_connection.close()
```

Also, we increased the connection queue by adjusting the backlog parameter to `listen_socket.listen(10)`.

This modification enables the operating system to buffer multiple incoming connection requests while the server handles current clients.

# Step 4: How to compare the performance?

To evaluate the efficiency of each implementation, we utilize a parallel workload generated by `curl`. By measuring the total execution time for all requests to be finalized, we can determine the processing throughput. This is achieved using a bash script that simulates 100 client requests with a concurrency level of 10.

Benchmark execution (100 total requests, 10 concurrent):

1) **Single-Threaded Performance Evaluation**

To begin, initialize the single-threaded server instance:
`python3 server.py`

In a separate terminal window, execute the benchmark command to send 100 requests with a concurrency of 10:

```
/usr/bin/time -p sh -c 'seq 1 100 | xargs -P10 -I{} curl -sS
--fail -o /dev/null http://localhost:9090/test.html'
```

seq 1 100: creates 100 test requests.
xargs -P10: runs up to 10 curl requests concurrently.
curl -sS: hides progress but displays errors.
--fail: makes curl report unsuccessful HTTP responses.

**Example timing output:**

```
real 2.11
user 0.32
sys 0.41
```

real value: This metric represents the total time elapsed to process the full batch of 100 requests.

2) **Multi-Threaded Performance Evaluation**

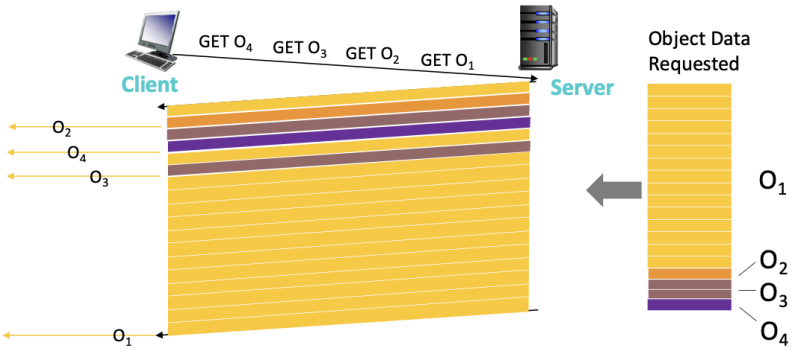
Initialize the concurrent server:
`python3 thread-server.py`
Execute the identical workload in a different terminal:

```
/usr/bin/time -p sh -c 'seq 1 100 | xargs -P10 -I{} curl -sS
--fail -o /dev/null http://localhost:9090/test.html'
```

```
real 0.19
user 0.36
sys 0.54
```

The results demonstrate that the multi-threaded server requires significantly less time to process the workload, achieving a completion time of 0.19 seconds compared to 2.11 seconds for the single-threaded version.

# Step 5: Mitigating the HOL blocking problem by using frames




HTTP/2: Objects divided into frames, frame transmission interleaved
*$O_2$, $O_3$, $O_4$ delivered quickly, $O_1$ slightly delayed*

We wrote two customized client and server handlers that implement delivery of multiple data sources using frames. This helps with mitigating the delay of large file transfers.

frame-server.py is a multithreaded TCP file server that uses a simplified framing protocol. It can transmit multiple requested files over one connection while interleaving their data to reduce application-level head-of-line blocking

Also as a side note, frame-server.py still supports the multi-thread functionality of thread-server.py.

The server expects a custom text request such as:

```
FILES 2
server.py
Test.html
```

```
        """
         For example, a request text might look like this:
         req text:    {FILES 2\n
                       big.txt\n
                       small.txt\n}
         """
```

FILES at the beginning of request indicate transmission of multiple files. The first line states the number of requested files. Each remaining line contains one file path. A blank line marks the end of the request.

The server continues receiving data until it finds the terminating blank line, then validates the entire request.

After parsing the request text, every requested file receives a stream ID, starting from 1. For example:

```
Stream 1: big.txt
Stream 2: small.txt
```

If a file is missing, the stream contains a 404 Not Found error. If the path attempts to leave the server directory, it contains a 403 Forbidden error.

File data is divided into payloads of no more than 512 bytes. Every frame contains a nine-byte header:

```
Response frame header (9 Bytes):
     Stream ID:       4 bytes
     Payload length:  4 bytes
     Flags:           1 byte

Payload of frame (up to 512 bytes)
     Payload:         0–512 bytes

"""Send one frame: stream ID + payload length + flags +
payload."""
```

The header is encoded with the `!IIB` structure format, which uses network byte order.

The flag values are:

```
0: normal data frame
1: end of stream
2: error
3: error and end of stream
```

Using round-robin, the server sends one frame from each active stream per round. It then returns to the first unfinished stream. This allows a small second file to finish before a large first file.

Respective Frames of each stream will be combined into complete files gradually on the client side and saved under /received.

# Step5: How to test:

Run the server using:

`python3 frame-server.py`

And client:

`python3 frame-client.py`

Two example files big.txt and small.txt are included to test the integrity of frames. You can use any number of files with different sizes writing file names as arguments:

`python3 frame-client.py big.txt small.txt`